Testen mit JUnit (JUnit-Basics)

Carsten Gips (HSBI)

TL;DR
In JUnit 4 und 5 werden Testmethoden mit Hilfe der Annotation `@Test` ausgezeichnet. Über die verschiedenen `assert*()`-Methoden kann das Testergebnis mit dem erwarteten Ergebnis verglichen werden und entsprechend ist der Test "grün" oder "rot". Mit den verschiedenen `assume*()`-Methoden kann dagegen geprüft werden, ob eventuelle Vorbedingungen für das Ausführen eines Testfalls erfüllt sind - anderenfalls wird der Testfall dann übersprungen.
Mit Hilfe von `@Before` und `@After` können Methoden gekennzeichnet werden, die jeweils vor jeder Testmethode und nach jeder Testmethode aufgerufen werden. Damit kann man seine Testumgebung auf- und auch wieder abbauen (JUnit 4).
Erwartete Exceptions lassen sich in JUnit 4 mit einem Parameter `expected` in der Annotation `@Test` automatisch prüfen: `@Test(expected=package.Exception.class)`. In JUnit 4 besteht die Möglichkeit, Testklassen zu Testsuiten zusammenzufassen und gemeinsam laufen zu lassen.

Videos

- VL JUnit Basics
- Demo assume() vs. assert()
- Demo Parametrisierte Tests mit JUnit4
- Demo Parametrisierte Tests mit JUnit5

Unterlagen

- PDF-Version

---

1 JUnit: Ergebnis prüfen

Klasse `org.junit.Assert` enthält diverse **statische** Methoden zum Prüfen:

```
// Argument muss true bzw. false sein
void assertTrue(boolean);
void assertFalse(boolean);

// Gleichheit im Sinne von equals()
void assertEquals(Object, Object);

// Test sofort fehlschlagen lassen
void fail();

...
```

2 To "assert" or to "assume"?

- Mit `assert*` werden Testergebnisse geprüft
  - Test wird ausgeführt

  - Ergebnis: OK, Failure, Error

- Mit `assume*` werden Annahmen über den Zustand geprüft
  - Test wird abgebrochen, wenn Annahme nicht erfüllt
  - Prüfen von Vorbedingungen: Ist der Test hier ausführbar/anwendbar?

Beispiel: junit4.TestAssume

3 Java: "assert"-Keyword

Neben den wohlbekannten `assert*`-Methoden aus JUnit gibt es auch direkt von Java ein etwas verstecktes `assert`-Keyword, mit dem man Annahmen über Zustände und Werte explizit ausdrücken kann:

```
1 public void foo() {
2     String bar = wuppie();
3     assert bar != null : "result of wuppie() must not be null";
4 }
```

Das `assert` besteht aus einer zu prüfenden Bedingung und einem String. Wenn die Bedingung erfüllt ist, läuft der Code einfach normal weiter. Anderenfalls wird ein `AssertionError` geworfen mit dem angegebenen String als Message.

Allerdings sind diese JVM-Assertions per Default **deaktiviert**. Man muss sie beim Aufruf manuell über die Option `-enableassertions` bzw. `-ea` (Kurzschreibweise) aktivieren (`java -ea main`)! Dies gilt auch beim Start über die IDE oder Gradle …

Wichtig: Die Assertions sind per Default deaktiviert und müssen erst manuell aktiviert werden. Außerdem wird bei Verletzung der Bedingung eine *unchecked exception* (ein Error) geworfen, der auf einen nicht korrigierbaren Programmzustand hindeutet.

1. Nutzen Sie das Java-`assert` deshalb nicht als Ersatz für das normale Prüfen von Parametern von **`public`** Methoden (also Methoden der Schnittstelle, die Ihre Kunden aufrufen).
2. Während der Entwicklungszeit kann das Java-`assert` aber ganz nützlich sein, weil Sie so interne Annahmen sichtbar und prüfbar machen (vorausgesetzt, Sie haben `-ea` aktiviert).

   Analog könnte ein Java-`assert` an Stellen eingebaut werden, die eigentlich nicht erreichbar sein sollten (etwa nach einer Dauerschleife oder in einem nicht erreichbaren **`default`**-Zweig in einem **`switch`**).
3. Bitte das Java-`assert` **nie** in einer JUnit-Testmethode statt der "richtigen" JUnit-`assert*` verwenden!

4 Setup und Teardown: Testübergreifende Konfiguration

```java
private Studi x;

@Before
public void setUp() { x = new Studi(); }

@Test
public void testToString() {
    // Studi x = new Studi();
    assertEquals(x.toString(), "Heinz (15cps)");
}
```

`@Before` wird **vor jeder** Testmethode aufgerufen
`@BeforeClass` wird **einmalig** vor allen Tests aufgerufen (**`static`**!)
`@After` wird **nach jeder** Testmethode aufgerufen
`@AfterClass` wird **einmalig** nach allen Tests aufgerufen (**`static`**!)

In JUnit 5 wurden die Namen dieser Annotationen leicht geändert:

JUnit 4	JUnit 5
`@Before`	`@BeforeEach`
`@After`	`@AfterEach`
`@BeforeClass`	`@BeforeAll`
`@AfterClass`	`@AfterAll`

5 Beispiel für den Einsatz von `@Before`

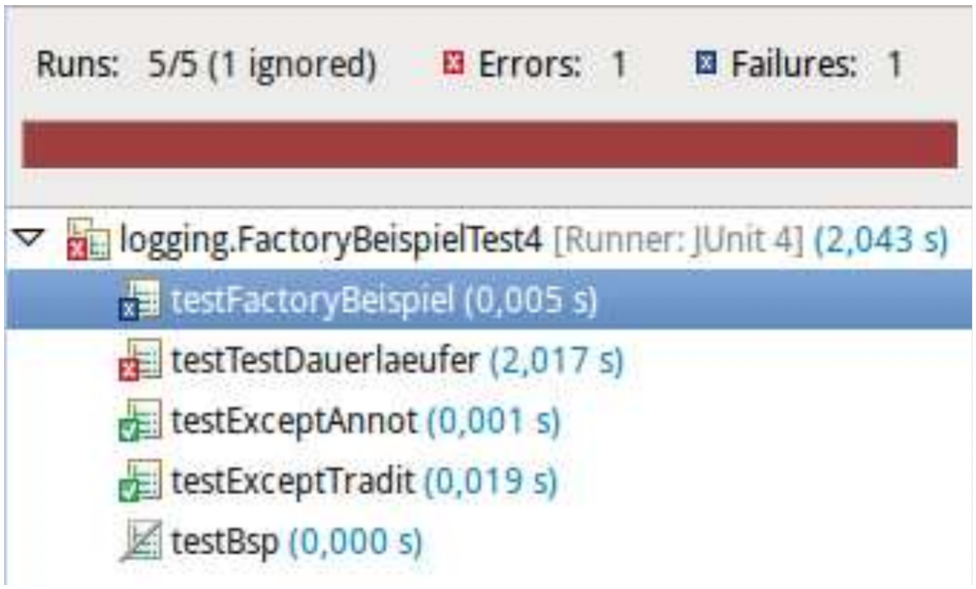
Annahme: **alle/viele** Testmethoden brauchen **neues** Objekt `x` vom Typ `Studi`

```java
private Studi x;

@Before
public void setUp() {
    x = new Studi("Heinz", 15);
}

@Test
public void testToString() {
    // Studi x = new Studi("Heinz", 15);
    assertEquals(x.toString(), "Name: Heinz, credits: 15");
}

@Test
public void testGetName() {
    // Studi x = new Studi("Heinz", 15);
    assertEquals(x.getName(), "Heinz");
}
```

6 Ignorieren von Tests

- Hinzufügen der Annotation `@Ignore`
- Alternativ mit Kommentar: `@Ignore("Erst im nächsten Release")`

```
@Ignore("Warum ignoriert")
@Test
public void testBsp() {
    Bsp x = new Bsp();
    assertTrue(x.isTrue());
}
```

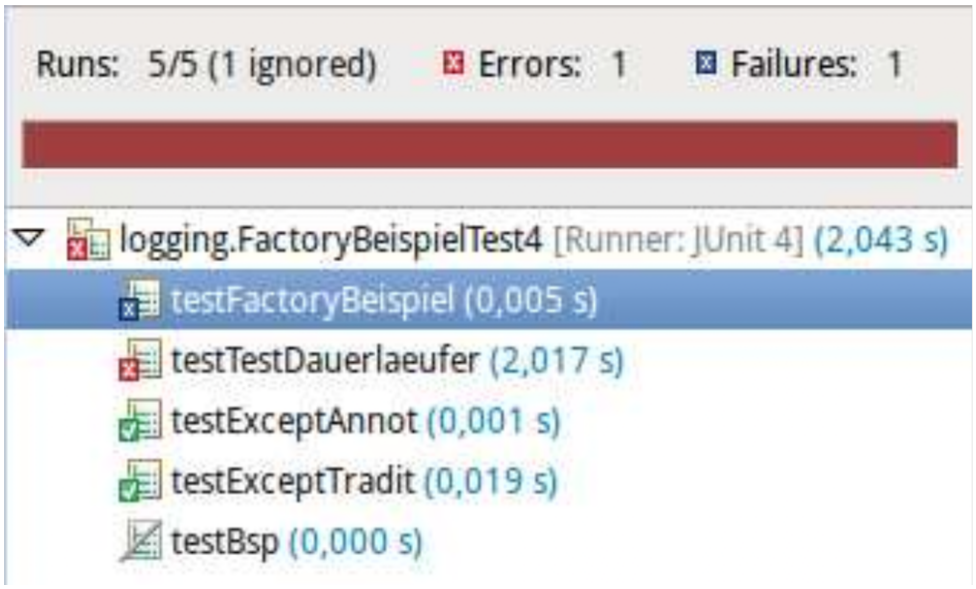


In JUnit 5 wird statt der Annotation `@Ignore` die Annotation `@Disabled` mit der selben Bedeutung verwendet. Auch hier lässt sich als Parameter ein String mit dem Grund für das Ignorieren des Tests hinterlegen.

7 Vermeidung von Endlosschleifen: Timeout

- Testfälle werden nacheinander ausgeführt
- Test mit Endlosschleife würde restliche Tests blockieren
- Erweitern der `@Test`-Annotation mit Parameter "`timeout`":
  => `@Test(timeout=2000)` (Zeitangabe in Millisekunden)

```
@Test(timeout = 2000)
void testTestDauerlaeufer() {
    while (true) { ; }
}
```



In JUnit 5 hat die Annotation `@Test` keinen `timeout`-Parameter mehr. Als Alternative bietet sich der Einsatz von `org.junit.jupiter.api.Assertions.assertTimeout` an. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
1  @Test
2  void testTestDauerlaeufer() {
3      assertTimeout(ofMillis(2000), () -> {
4          while (true) { ; }
5      });
6  }
```

(Beispiel von oben mit Hilfe von JUnit 5 formuliert)

8 Test von Exceptions: Expected

Traditionelles Testen von Exceptions mit **`try`** und **`catch`**:

```
1  @Test
2  public void testExceptTradit() {
3      try {
4          int i = 0 / 0;
5          fail("keine ArithmeticException ausgeloest");
6      } catch (ArithmeticException aex) {
7          assertNotNull(aex.getMessage());
8      } catch (Exception e) {
9          fail("falsche Exception geworfen");
10     }
11 }
```

Der `expected`-Parameter für die `@Test`-Annotation in JUnit 4 macht dies deutlich einfacher: `@Test(expected = MyException.class)` => Test scheitert, wenn diese Exception **nicht** geworfen wird

```
@Test(expected = java.lang.ArithmeticException.class)
public void testExceptAnnot() {
    int i = 0 / 0;
}
```

In JUnit 5 hat die Annotation `@Test` keinen `expected`-Parameter mehr. Als Alternative bietet sich der Einsatz von `org.junit.jupiter.api.Assertions.assertThrows` an. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
@Test
public void testExceptAnnot() {
    assertThrows(java.lang.ArithmeticException.class, () -> {
        int i = 0 / 0;
    });
}
```

(Beispiel von oben mit Hilfe von JUnit 5 formuliert)

9 “Given - When - Then”-Mantra

Aus dem Behavior-driven development stammt die Strukturierung nach den Punkten “given - when - then” (oft auch als *“given - when - then”-Mantra* bezeichnet).

Betrachten Sie noch einmal die Schnittstelle der Studi-Klasse:

```
public class Studi {
    public String getName();
    public void setName(String name);
    public int getCredits();
    public void addToCredits(int credits);
}
```

Dafür wurde ein Test geschrieben:

```
class StudiTest {
    @Test
    public void testStudi() {
        Studi s = new Studi();
        int cps = 2;
        s.addToCredits(cps);
        assertEquals(cps, s.getCredits());
    }
}
```

Dieser Code ist in seiner Absicht nicht sofort verständlich. Es fällt auf, dass

1. am Anfang eine Art Setup des Tests vorgenommen wird und das Testobjekt initialisiert wird (“**given**”).
2. Dann wird die zu untersuchende Aktion ausgeführt (“**when**”), gefolgt von
3. einem Vergleich des tatsächlichen mit dem erwarteten Ergebnis (“**then**”).

Diese gedachte Struktur kann (und sollte!) man mit entsprechenden Kommentaren auch sichtbar machen:

```
class StudiTest {
    @Test
    public void testStudi() {
        // given
        Studi s = new Studi();
        int cps = 2;

        // when
        s.addToCredits(cps);

        // then
        assertEquals(cps, s.getCredits());
    }
}
```

In Testframeworks wie Spock oder Cucumber ist dies sogar bereits in die Sprache (eine DSL zum Testen) eingebaut! Einen schönen Blog zum Thema finden Sie hier: Spock testing framework versus JUnit.

Weiterhin könnte (und sollte) man die implizit getroffenen Annahmen über das SUT für alle sichtbar machen:

```
class StudiTest {
    @Test
    public void testStudi() {
        // given
        Studi s = new Studi();
        int cps = 2;
        assumeTrue(s.getCredits() == 0, "initial credits should be 0");

        // when
        s.addToCredits(cps);

        // then
        assertEquals(cps, s.getCredits());
    }
}
```

Der Test würde ohnehin fehlschlagen, wenn ein neues `Studi`-Objekt mit einem anderen Wert für die Credits initialisiert würde. Aber so zeigt das `assume` direkt unsere (bisher) implizite Annahme sichtbar an, und bei einer Verletzung dieser Annahme würde der Testfall mit einer entsprechenden Mitteilung nicht ausgeführt.

Oft wird noch das “given - when - then”-Mantra auch auf die Methodennamen der Testmethoden übertragen:[1]

```
class StudiTest {
    @Test
    public void
        GivenNewStudi_WhenAddingCredits_ThenCreditsCountChangesAccordingly() {
        // given
        Studi s = new Studi();
        int cps = 2;
        assumeTrue(s.getCredits() == 0, "initial credits should be 0");

        // when
        s.addToCredits(cps);

```

[1] Naja, ein kläglicher Versuch. Namen sind eines der schwierigen Probleme in der Informatik.

```
        // then
        assertEquals(cps, s.getCredits());
    }
}
```

Eine schöne Erklärung finden Sie im Blog The subtle Art of Java Test Method Naming.

… Und nun könnte man sich fragen, warum man das Erhöhen von Credits nur für ein *neues* `Studi`-Objekt testet und nicht auch für andere Zustände des `Studi`-Objekts? … => Parametrisierte Tests!

10 Parametrisierte Tests

Manchmal möchte man den selben Testfall mehrfach mit anderen Werten (Parametern) durchführen.

```
class Sum {
    public static int sum(int i, int j) {
        return i + j;
    }
}

class SumTest {
    @Test
    public void testSum() {
        Sum s = new Sum();
        assertEquals(s.sum(1, 1), 2);
    }
    // und mit (2,2, 4), (2,2, 5), ...????
}
```

Prinzipiell könnte man dafür entweder in einem Testfall eine Schleife schreiben, die über die verschiedenen Parameter iteriert. In der Schleife würde dann jeweils der Aufruf der zu testenden Methode und das gewünschte Assert passieren. Alternativ könnte man den Testfall entsprechend oft duplizieren mit jeweils den gewünschten Werten.

Beide Vorgehensweisen haben Probleme: Im ersten Fall würde die Schleife bei einem Fehler oder unerwarteten Ergebnis abbrechen, ohne dass die restlichen Tests (Werte) noch durchgeführt würden. Im zweiten Fall bekommt man eine unnötig große Anzahl an Testmethoden, die bis auf die jeweiligen Werte identisch sind (Code-Duplizierung).

10.1 Parametrisierte Tests mit JUnit 4

JUnit 4 bietet für dieses Problem sogenannte “parametrisierte Tests” an. Dafür muss eine Testklasse in JUnit 4 folgende Bedingungen erfüllen:

1. Die Testklasse wird mit der Annotation `@RunWith(Parameterized.class)` ausgezeichnet.
2. Es muss eine öffentliche statische Methode geben mit der Annotation `@Parameters`. Diese Methode liefert eine Collection zurück, wobei jedes Element dieser Collection ein Array mit den Parametern für einen Durchlauf der Testmethoden ist.

3. Die Parameter müssen gesetzt werden. Dafür gibt es zwei Varianten:
   - (A) Für jeden Parameter gibt es ein öffentliches Attribut. Diese Attribute müssen mit der Annotation `@Parameter` markiert sein und können in den Testmethoden normal genutzt werden. JUnit sorgt dafür, dass für jeden Eintrag in der Collection aus der statischen `@Parameters`-Methode diese Felder gesetzt werden und die Testmethoden aufgerufen werden.
   - (B) Alternativ gibt es einen Konstruktor, der diese Werte setzt. Die Anzahl der Parameter im Konstruktor muss dabei exakt der Anzahl (und Reihenfolge) der Werte in jedem Array in der von der statischen `@Parameters`-Methode gelieferten Collection entsprechen. Der Konstruktor wird für jeden Parametersatz einmal aufgerufen und die Testmethoden einmal durchgeführt.

Letztlich wird damit das Kreuzprodukt aus Testmethoden und Testdaten durchgeführt.

10.1.1 (A) Parametrisierte Tests: Konstruktor (JUnit 4)

```java
@RunWith(Parameterized.class)
public class SumTestConstructor {
    private final int s1;
    private final int s2;
    private final int erg;

    public SumTestConstructor(int p1, int p2, int p3) { s1 = p1;  s2 = p2;  erg = p3; }

    @Parameters
    public static Collection<Object[]> values() {
        return Arrays.asList(new Object[][] { { 1, 1, 2 }, { 2, 2, 4 }, { 2, 2, 5 } });
    }

    @Test
    public void testSum() {
        assertEquals(Sum.sum(s1, s2), erg);
    }
}
```

10.1.2 (B) Parametrisierte Tests: Parameter (JUnit 4)

```java
@RunWith(Parameterized.class)
public class SumTestParameters {

    @Parameter(0)  public int s1;
    @Parameter(1)  public int s2;
    @Parameter(2)  public int erg;

    @Parameters
    public static Collection<Object[]> values() {
        return Arrays.asList(new Object[][] { { 1, 1, 2 }, { 2, 2, 4 }, { 2, 2, 5 } });
    }

```

```
13      @Test
14      public void testSum() {
15          assertEquals(Sum.sum(s1, s2), erg);
16      }
17  }
```

Beispiel: junit4.SumTestConstructor, junit4.SumTestParameters

10.2 Parametrisierte Tests mit JUnit 5

In JUnit 5 werden parametrisierte Tests mit der Annotation `@ParameterizedTest` gekennzeichnet (statt mit `@Test`).

Mit Hilfe von `@ValueSource` kann man ein einfaches Array von Werten (Strings oder primitive Datentypen) angeben, mit denen der Test ausgeführt wird. Dazu bekommt die Testmethode einen entsprechenden passenden Parameter:

```
1  @ParameterizedTest
2  @ValueSource(strings = {"wuppie", "fluppie", "foo"})
3  void testWuppie(String candidate) {
4      assertTrue(candidate.equals("wuppie"));
5  }
```

Alternativ lassen sich als Parameterquelle u.a. Aufzählungen (`@EnumSource`) oder Methoden (`@MethodSource`) oder auch Komma-separierte Daten (`@CsvSource`) angeben.

Das obige Beispiel aus JUnit 4.x könnte mit Hilfe von `@CsvSource` so in JUnit 5.x umgesetzt werden:

```
1  public class SumTest {
2      @ParameterizedTest
3      @CsvSource(textBlock = """
4              # s1,  s2,  s1+s2
5              0,     0,   0
6              10,    0,   10
7              0,     11,  11
8              -2,    10,  8
9              """)
10     public void testSum(int s1, int s2, int erg) {
11         assertEquals(Sum.sum(s1, s2), erg);
12     }
13 }
```

Beispiel: junit5.TestValueSource, junit5.TestMethodSource

11 Testsuiten: Tests gemeinsam ausführen (JUnit 4)

Eclipse: `New` > `Other` > `Java` > `JUnit` > `JUnit Test Suite`

```
1  import org.junit.runner.RunWith;
2  import org.junit.runners.Suite;
3  import org.junit.runners.Suite.SuiteClasses;
```

```

@RunWith(Suite.class)
@SuiteClasses({
    // Hier kommen alle Testklassen rein
    PersonTest.class,
    StudiTest.class
})

public class MyTestSuite {
    // bleibt leer!!!
}
```

12 Testsuiten mit JUnit 5

In JUnit 5 gibt es zwei Möglichkeiten, Testsuiten zu erstellen:

- `@SelectPackages`: Angabe der Packages, die für die Testsuite zusammengefasst werden sollen
- `@SelectClasses`: Angabe der Klassen, die für die Testsuite zusammengefasst werden sollen

```
@RunWith(JUnitPlatform.class)
@SelectClasses({StudiTest5.class, WuppieTest5.class})
public class MyTestSuite5 {
    // bleibt leer!!!
}
```

Zusätzlich kann man beispielsweise mit `@IncludeTags` oder `@ExcludeTags` Testmethoden mit bestimmten Tags einbinden oder ausschließen. Beispiel: Schließe alle Tests mit Tag "develop" aus: `@ExcludeTags("develop")`. Dabei wird an den Testmethoden zusätzlich das Tag `@Tag` verwendet, etwas `@Tag("develop")`.

Achtung: Laut der offiziellen Dokumentation (Abschnitt "4.4.4. Test Suite") gilt zumindest bei der Selection über `@SelectPackages` der Zwang zu einer Namenskonvention: Es werden dabei nur Klassen gefunden, deren Name mit `Test` beginnt oder endet! Weiterhin werden Testsuites mit der Annotation `@RunWith(JUnitPlatform.class)` **nicht** auf der "JUnit 5"-Plattform ausgeführt, sondern mit der JUnit 4-Infrastuktur!

13 Best Practices

1. Eine Testmethode behandelt exakt eine Idee/ein Szenario (einen Testfall). Das bedeutet auch, dass man in der Regel nur ein bis wenige `assert*` pro Testmethode benutzt.

   (Wenn man verschiedene Ideen in eine Testmethode kombiniert, wird der Testfall unübersichtlicher und auch auch schwerer zu warten.

   Außerdem können so leichter versteckte Fehler auftreten: Das erste oder zweite oder dritte `assert*` schlägt fehl - und alle dahinter kommenden `assert*` werden nicht mehr ausgewertet!)

2. Wenn die selbe Testidee mehrfach wiederholt wird, sollte man diese Tests zu einem parametrisierten Test zusammenfassen.

   (Das erhöht die Lesbarkeit drastisch - und man läuft auch nicht in das Problem der Benennung der Testmethoden.)

3. Es wird nur das Verhalten der öffentlichen Schnittstelle getestet, nicht die inneren Strukturen einer Klasse oder Methode.

   (Es ist verlockend, auch private Methoden zu testen und in den Tests auch die Datenstrukturen o.ä. im Blick zu behalten und zu testen. Das führt aber zu sehr "zerbrechlichen" (*brittle*) Tests: Sobald sich etwas an der inneren Struktur ändert, ohne dass sich das von außen beobachtbare Verhalten ändert und also die Klasse/Methode immer noch ordnungsgemäß funktioniert, gehen all diese "internen" Tests kaputt. Nicht ohne Grund wird in der objektorientierten Programmierung mit Kapselung (Klassen, Methoden, ...) gearbeitet.)

4. Von Setup- und Teardown-Methoden sollte eher sparsam Gebrauch gemacht werden.

   (Normalerweise folgen wir in der objektorientierten Programmierung dem DRY-Prinzip (Don't repeat yourself). Entsprechend liegt es nahe, häufig benötigte Elemente in einer Setup-Methode zentral zu initialisieren und ggf. in einer Teardown-Methode wieder freizugeben.

   Das führt aber speziell bei Unit-Tests dazu, dass die einzelnen Testmethoden schwerer lesbar werden: Sie hängen von einer gemeinsamen, zentralen Konfiguration ab, die man üblicherweise nicht gleichzeitig mit dem Code der Testmethode sehen kann (begrenzter Platz auf der Bildschirmseite).

   Wenn nun in einem oder vielleicht mehreren Testfällen der Wunsch nach einer leicht anderen Konfiguration auftaucht, muss man die gemeinsame Konfiguration entsprechend anpassen bzw. erweitern. Dabei muss man dann aber *alle* anderen Testmethoden mit bedenken, die ja ebenfalls von dieser Konfiguration abhängen! Das führt in der Praxis dann häufig dazu, dass die gemeinsame Konfiguration sehr schnell sehr groß und verschachtelt und entsprechend unübersichtlich wird.

   Jede Änderung an dieser Konfiguration kann leicht einen oder mehrere Testfälle kaputt machen (man hat ja i.d.R. nie alle Testfälle gleichzeitig im Blick), weshalb man hier unbedingt mit passenden `assume*` arbeiten muss - aber dann kann man eigentlich auch stattdessen die Konfiguration direkt passend für den jeweiligen Testfall in der jeweiligen Testmethode erledigen!)

5. Wie immer sollten auch die Namen der Testmethoden klar über ihren Zweck Auskunft geben.

   (Da Tests oft auch als "ausführbare Dokumentation" betrachtet werden, ist eine sinnvolle Benamung besonders wichtig. Oft werden hier deshalb Ausnahmen von den üblichen Java-Konventionen erlaubt. Man findet häufig das aus dem Behavior-driven development bekannte "given - when - then"-Mantra. Siehe auch The subtle Art of Java Test Method Naming und auch Spock testing framework versus JUnit.

   Der Präfix "test" für Testmethoden wird seit JUnit 4.x nicht mehr benötigt, aber dennoch ist es in vielen Projekten Praxis, diesen Präfix beizubehalten - damit kann man in der Package-Ansicht in der IDE leichter zwischen den "normalen" und den Testmethoden unterscheiden. Das ist analog zum Suffix "Test" für die Klassennamen der Testklassen ...)

Diese Erfahrungen werden ausführlich in (Winters, Manshreck, und Wright 2020, pp. 231-256) diskutiert.

Eine lesenswerte Diskussion von "Anti-Pattern" beim Testen finden Sie im Blog Software Testing Anti-patterns.

14 Wrap-Up

JUnit als Framework für (Unit-) Tests; hier JUnit 4 (mit Ausblick auf JUnit 5)

- Testmethoden mit Annotation `@Test`
- `assert` (Testergebnis) vs. `assume` (Testvorbedingung)
- Aufbau der Testumgebung `@Before`
- Abbau der Testumgebung `@After`
- Steuern von Tests mit `@Ignore` oder `@Test(timout=XXX)`
- Exceptions einfordern mit `@Test(expected=package.Exception.class)`
- Tests zusammenfassen zu Testsuiten

15 Zum Nachlesen

- vogella GmbH (2021)
- The JUnit Team (2022)
- Kleuker (2019)
- Osherove (2014)
- Spillner und Linz (2012)
- Thies, Noelke, und Ungerc (o. J.)

Lernziele

- k3: Ich kann Testergebnisse prüfen
- k2: Ich kenne den Unterschied zwischen assert und assume
- k3: Ich kann vor/nach jedem Test bestimmten Code ausführen
- k2: Ich habe verstanden, warum @Before und @After sparsam einzusetzen sind
- k3: Ich kann die Ausführung von Tests steuern, beispielsweise Tests ignorieren oder mit zeitlicher Begrenzung ausführen
- k3: Ich kann das Auftreten von Exceptions prüfen
- k3: Ich kann Tests zu Testsuiten zusammenfassen

Challenges

Setup und Teardown

Sie haben in den Challenges in “Intro SW-Test” erste JUnit-Tests für die Klasse `MyList<T>` implementiert.
Wie müssten Sie Ihre JUnit-Tests anpassen, wenn Sie im obigen Szenario Setup- und Teardown-Methoden einsetzen würden?

Testmethoden

Betrachten Sie den folgenden Code. Was fällt Ihnen auf?

```
public class Studi {
    public int getCredits();
    public void addToCredits(int credits);

    @Test
    public void testStudi() {
        Studi s = new Studi();
        s.addToCredits(2);
        assertEquals(2, s.getCredits());
    }
}
```

Parametrisierte Tests

Betrachten Sie die folgende einfache Klasse `MyMath`:

```
public class MyMath {
    public static String add(String s, int c) {
        return s.repeat(c);
    }
}
```

Beim Testen der Methode `MyMath#add` fällt auf, dass man hier immer wieder den selben Testfall mit lediglich anderen Werten ausführt - ein Fall für parametrisierte Tests.
Schreiben Sie mit Hilfe von JUnit (4.x oder 5.x) einige parametrisierte Unit-Tests für die Methode `MyMath#add`.

Quellen

Kleuker, S. 2019. *Qualitätssicherung durch Softwaretests*. Springer Vieweg. https://doi.org/10.1007/978-3-658-24886-4.

Osherove, R. 2014. *The Art of Unit Testing*. Manning.

Spillner, A., und T. Linz. 2012. *Basiswissen Softwaretest*. 5. Aufl. dpunkt.

The JUnit Team. 2022. „JUnit 5“. 2022. https://junit.org/.

Thies, A., C. Noelke, und Ungerc. o. J. „Einführung in JUnit“. Fernuniversität in Hagen. Zugegriffen 14. April 2020. https://wiki.fernuni-hagen.de/eclipse/index.php/EinfÃijhrung_in_JUnit.

vogella GmbH. 2021. „JUnit 5 Tutorial - Learn How to Write Unit Tests“. 2021. https://www.vogella.com/tutorials/JUnit/article.html.

Winters, T., T. Manshreck, und H. Wright. 2020. *Software Engineering at Google: Lessons Learned from Programming Over Time*. O’Reilly.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 660b2f4 (lecture: add blogs on software testing (JUnit Basics), 2025-05-15)